

NLP

Analyse de la satisfaction client dans la cosmétique

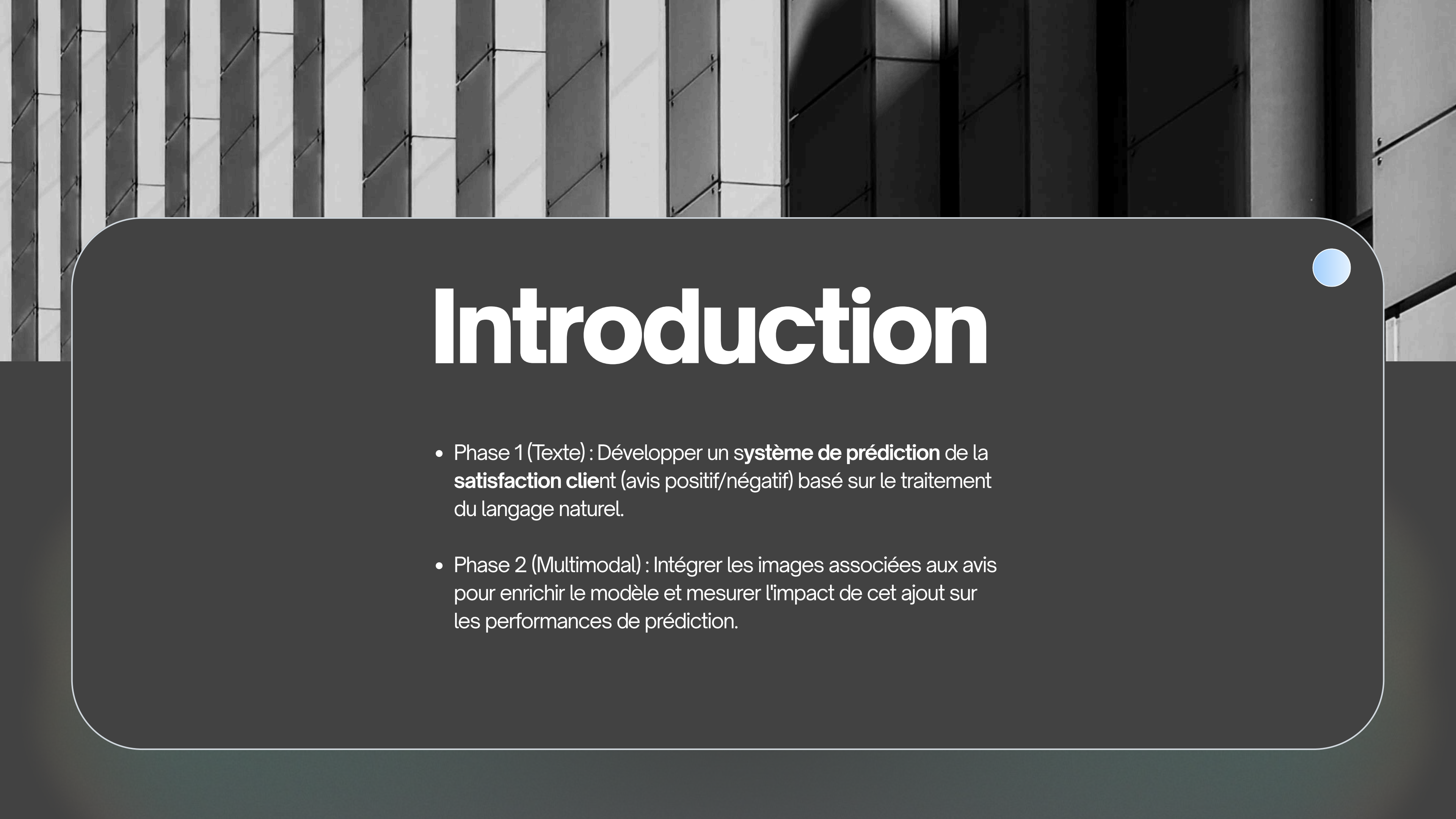
ING3-DS

2025-26

Encadrant : Nistor Grozavu

Sommaire

- 01** Introduction
- 02** Analyse du dataset
- 03** Prétraitement et transformation
- 04** Techniques d'Embedding
- 05** Système de classification
- 06** Approche Multimodale
- 07** Analyse des Résultats
- 08** Analyse des Résultats



Introduction

- Phase 1 (Texte) : Développer un **système de prédiction** de la **satisfaction client** (avis positif/négatif) basé sur le traitement du langage naturel.
- Phase 2 (Multimodal) : Intégrer les images associées aux avis pour enrichir le modèle et mesurer l'impact de cet ajout sur les performances de prédiction.

Présentation du dataset

Amazon “All_Beauty”

- Contient 701 528 avis clients sur des produits cosmétiques
- Titre (title), le corps de l’avis (text) et la note de l’utilisateur (rating)
- Contient, pour chaque avis utilisateur, l’url de l’image du produit commenté/noté

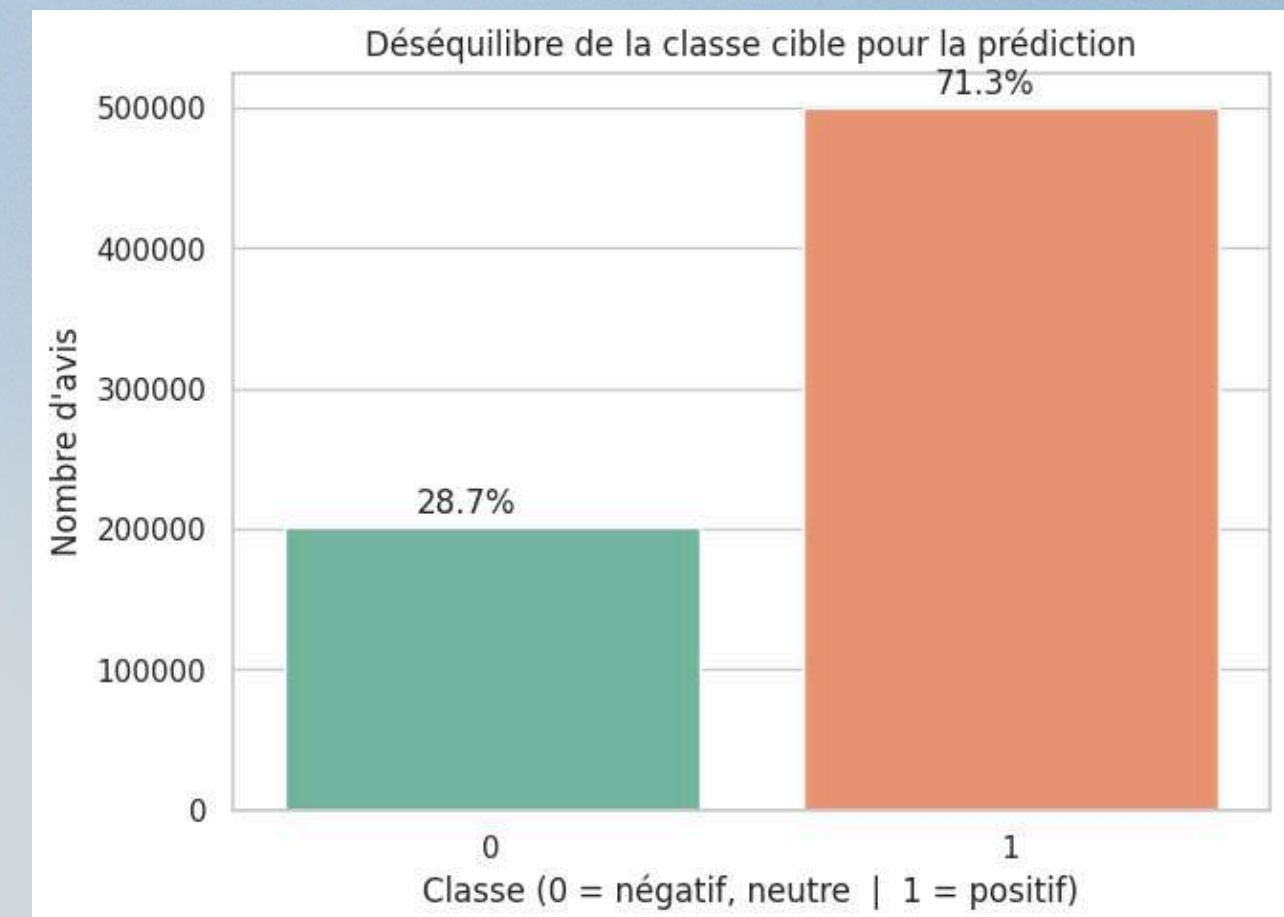
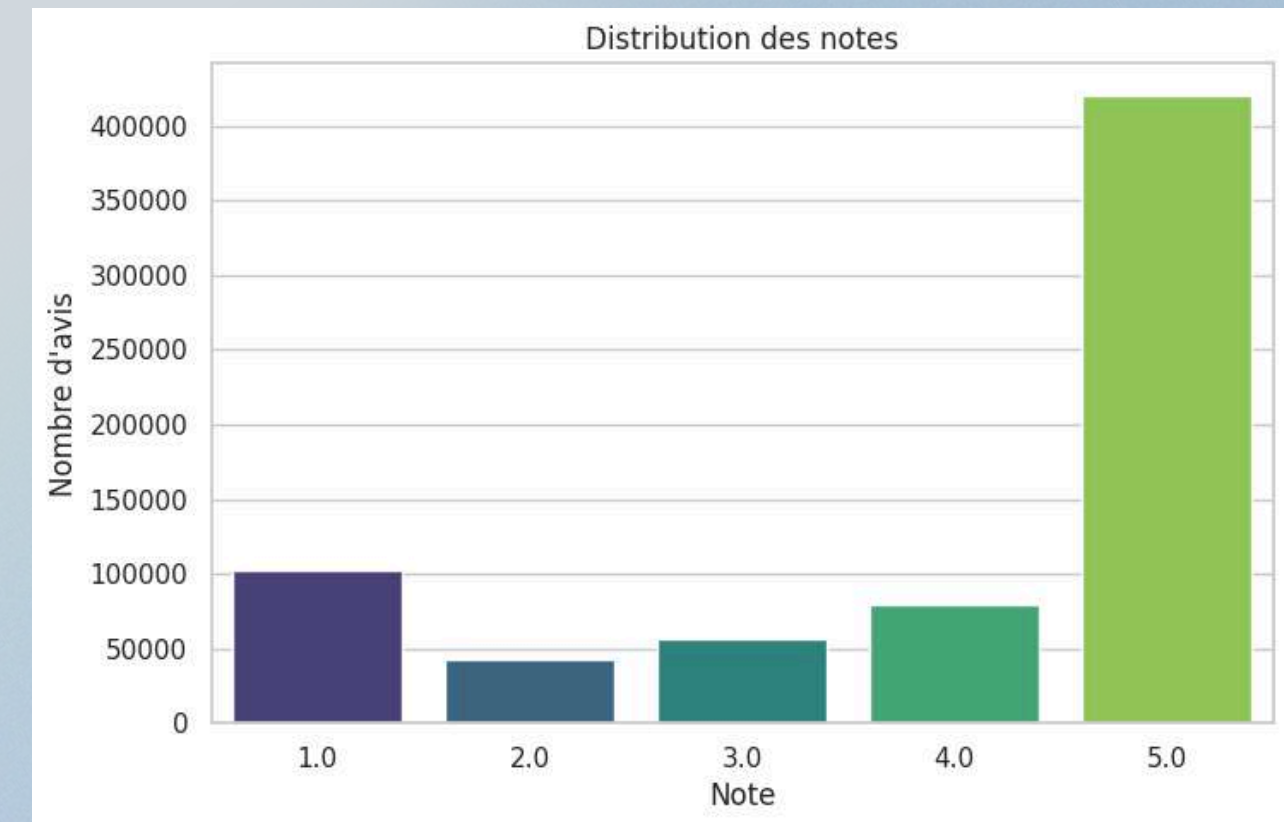


Analyse du dataset

71,3% D'avis positifs (note ≥ 4)

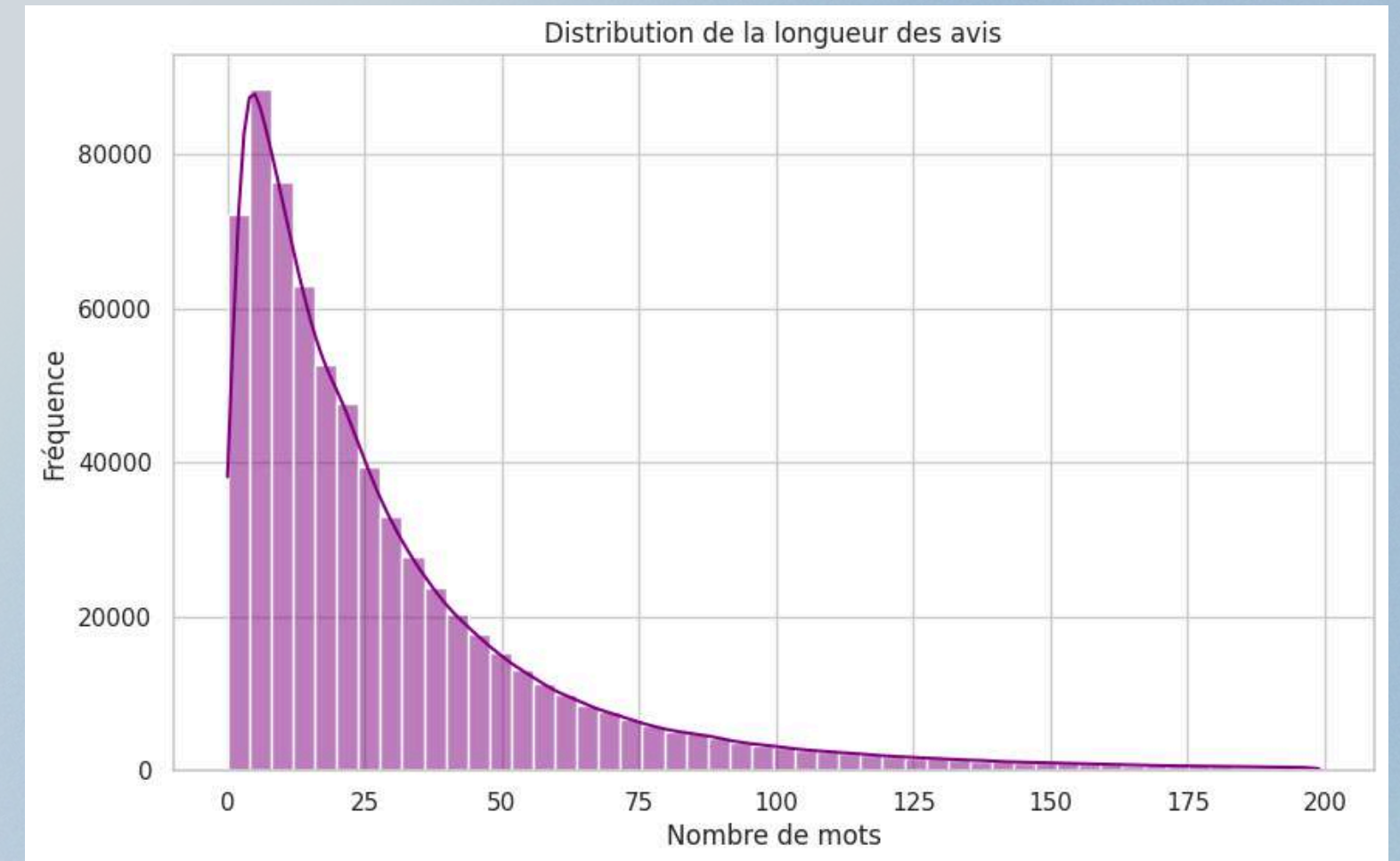
28,7% D'avis neutre/négatifs (note < 4)

- Forte asymétrie des notes brutes, avec une domination écrasante des 5 étoiles.
- La classe cible "Positif" représente 71,3% du dataset contre 28,7% pour la classe "Négatif/Neutre".
- L'Accuracy n'est pas une métrique fiable. Il y a donc nécessité d'utiliser le F1-Score ou l'AUC-ROC et d'envisager des pondérations de classes (class weights).



Analyse du dataset

- Constat : Distribution très asymétrique. La majorité des avis sont très courts (pic autour de 10 à 20 mots).
- Enjeu ML : Le modèle devra extraire le sentiment à partir de textes très condensés (peu de contexte). Cela orientera nos choix de padding et de troncature lors de l'étape d'Embedding.



Prétraitement & transformation

Nettoyage

- Concaténation des colonnes title et text
- Retrait de la ponctuation et des chiffres

"Analyse des données en 2024 !!!"



"analyse des données"

Filtrage

- Suppression des stop words et des termes de moins de 3 caractères

"analyse des données est très importante"



"analyse des données importante"

Stemming

- Approche heuristique par troncature des suffixes via CountVectorizer

"manger, mangeons, mangera"



"mang"

Lemmatisation

- Analyse morphologique pour extraire la racine canonique

"mangeons, mangé, mangerai"

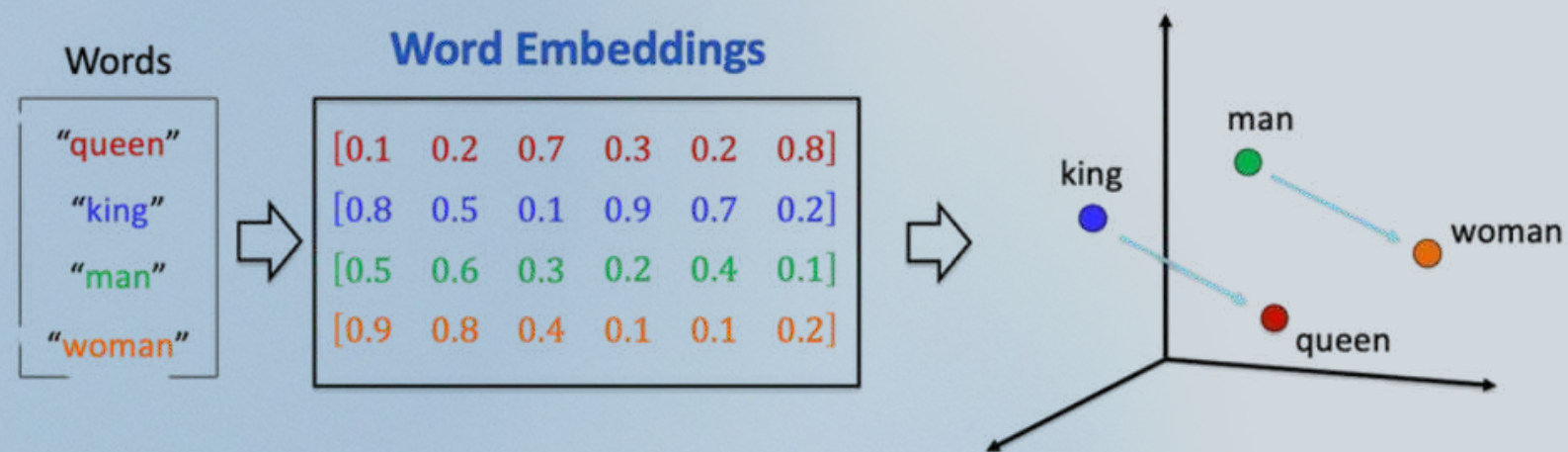


"manger"

Word Embeddings

Obtenir des représentation denses :
passage de matrices creuses
(BOW/TF-IDF) à des vecteurs de
faible dimension.

Le but étant de capturer la proximité
sémantique



Word2Vec

- Apprend la représentation des mots en prédisant leur contexte local
- Capture finement les relations sémantiques et syntaxiques (ex: l'opération mathématique Roi - Homme + Femme = Reine devient possible)

Document2Vec

- Ajoute un "vecteur d'identifiant de document" au processus de Word2Vec pour capturer le contexte global de la phrase ou du paragraphe entier
- Surmonte la limite de GloVe et Word2Vec qui obligent à faire une simple moyenne des mots pour obtenir le vecteur final d'un document.

Glove

- S'appuie sur la co-occurrence statistique globale des mots dans l'ensemble du corpus, plutôt que sur des fenêtres locales

Transformers : Sentence-BERT

- Modèle utilisé : all-MiniLM-L6-v2.
- Contrairement aux embeddings classiques, BERT comprend l'ordre des mots et les nuances de sens.
- Un vecteur unique de 384 dimensions par avis, prêt pour le ML

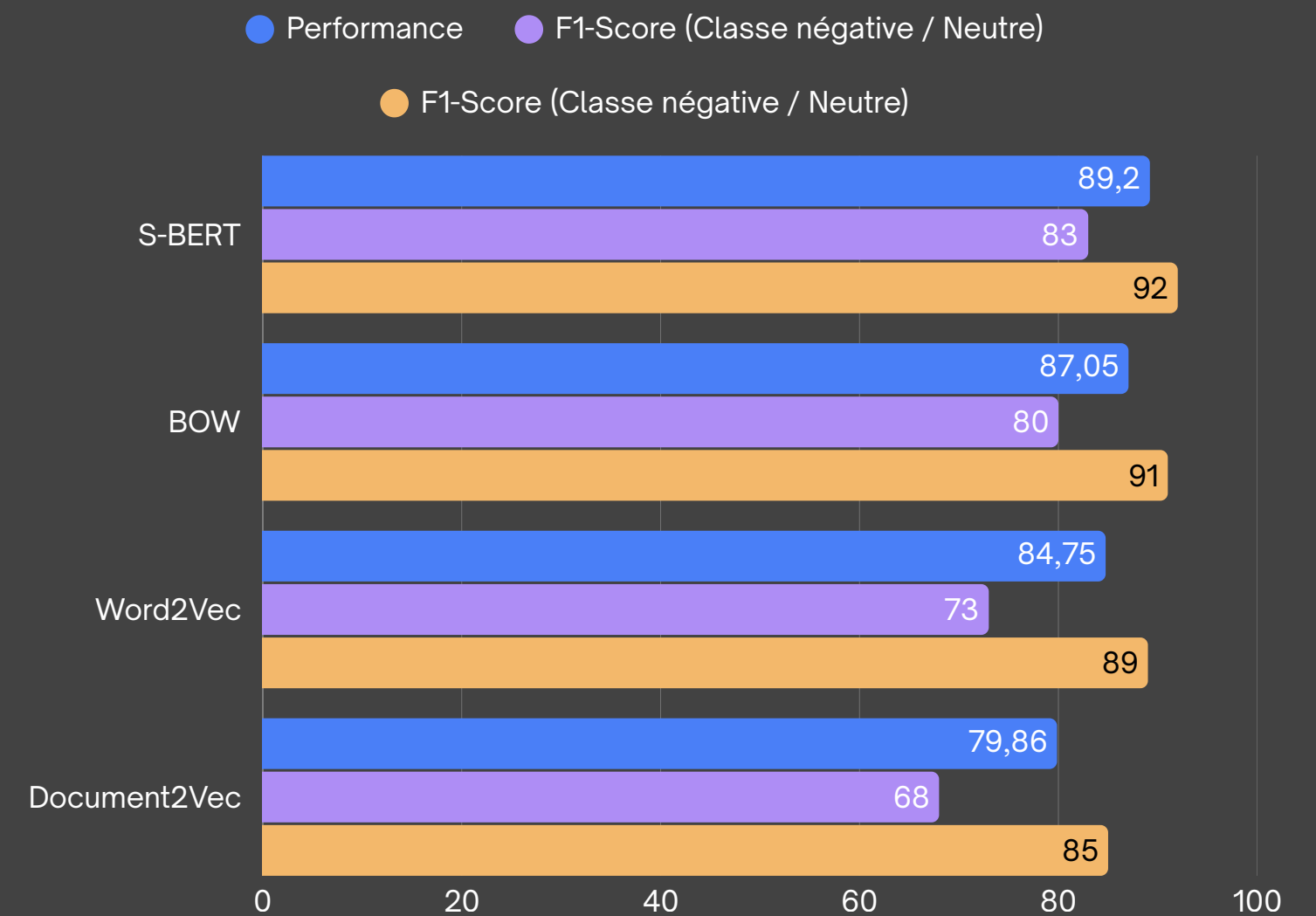
Classification et Modélisation

- Séparation des données : Partitionnement 80% (Entraînement) / 20% (Test) pour garantir la capacité de généralisation.
- Baseline Statistique : Régression Logistique avec régularisation, choisie pour sa rapidité et son efficacité sur des vecteurs denses.
- Approche Deep Learning : Réseau de neurones LSTM (Long Short-Term Memory) pour capturer les dépendances séquentielles et le contexte long des textes.
- Gestion du déséquilibre (Imbalance) : utilisation du paramètre `class_weight='balanced'` pour compenser le biais des 71,3% d'avis positifs identifiés lors de l'EDA.

Résultats

- Sentence-BERT (SBERT) : 89,20 % – Leader incontesté grâce à la **compréhension contextuelle bidirectionnelle**.
- Bag of Words (BOW) : 87,05 % – Très efficace sur les avis courts grâce aux **mots-clés polarisés**.
- Word2Vec : 84,75 % – Bonne capture sémantique mais limité par l'aspect **statique** des vecteurs.
- Document2Vec : 79,86 % – Difficulté à généraliser sur des avis très concis.

La richesse de l'embedding (SBERT) est le facteur déterminant de la performance, surpassant les méthodes fréquentielles et statiques.



Approche Multimodale

Hypothèse : l'ajout des images à notre système de classification (i.e. le passage à un système multimodale), devrait augmenter la performance de notre classifieur

Etape 1 : Feature extraction

Utilisation d'un CNN pré-entraîné (ResNet50)

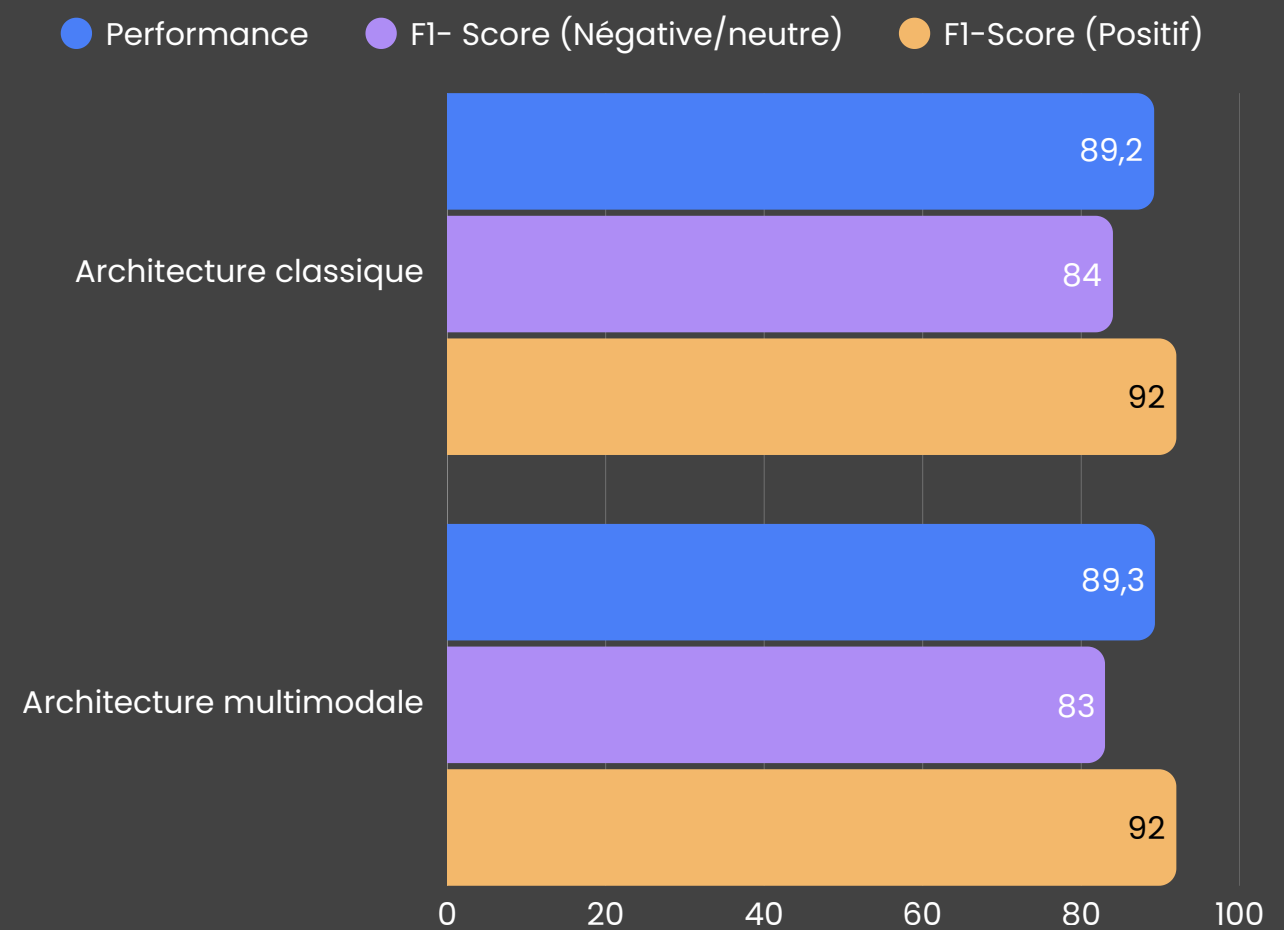
Etape 2 : Stratégie de fusion

Normalisation L2 : Étape indispensable pour ramener les vecteurs texte et image à la même échelle et éviter que les poids de ResNet soient prépondérants.

Résultats

- SBERT (Texte seul) : 89,20 %
- Multimodal (Texte + Image) : 89,30 %
- Gain de performance : +0,10 %

L'image apporte un signal complémentaire ("Features" produit) qui affine la décision du classifieur sur les cas ambigus.



BERT

- BERT est un modèle Transformer de type encoder-only.

Application direct sur un avis Amazon

Données

Avis Amazon : "This nice smells good"

Tokens:
["this", "nice", "smells", "good"]

$d_{\text{model}} = 4$

→ chaque token est représenté par un vecteur de 4 nombres

$d_k = 4$

→ les vecteurs Query, Key et Value auront aussi 4 dimensions

Étape 1 – Matrice d'entrée X

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \begin{array}{l} \rightarrow \text{"this"} \\ \rightarrow \text{"nice"} \\ \rightarrow \text{"smells"} \\ \rightarrow \text{"good"} \end{array}$$

Étape 2 – Matrices de projection

$$W^K = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \quad W^Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad W^V = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$$W^Q, W^K, W^V \in \mathbb{R}^{(d_{\text{model}} \times d_k)} \text{ Avec : } d_{\text{model}} = 4; d_k = 4$$

W^Q Crée les Query → ce que chaque token cherche dans les autres tokens

W^K Crée les Key → ce que chaque token propose comme information

W^V Crée les Value → l'information réellement transmise après attention



- Ce sont des paramètres appris (poids du modèle) qui sont initialisés aléatoirement
- Apprises avec backpropagation + Adam

Chaque matrice transforme les embeddings d'entrée X :

$$Q = XW^Q; K = XW^K; V = XW^V$$



Étape 3- Calculs de Q,K et V

$Q = XW^Q; K = XW^K; V = XW^V$

$Q = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 2 & 0 & 2 & 1 \\ 0 & 2 & 1 & 2 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 2 & 2 \end{bmatrix} \quad V = \begin{bmatrix} 2 & 0 & 2 & 1 \\ 0 & 2 & 1 & 2 \\ 1 & 1 & 2 & 2 \\ 1 & 1 & 1 & 1 \end{bmatrix}$

Étape 4- Matrice des scores d'attention

$QK^T / \sqrt{d_k}$ → Chaque élément (i,j) = combien le token i "regarde" le token j

$\frac{1}{\sqrt{4}}QK^T = \begin{bmatrix} 1.5 & 0.5 & 1.5 & 1.0 \\ 0.5 & 1.5 & 1.5 & 2.0 \\ 1.5 & 1.5 & 2.0 & 2.0 \\ 1.0 & 2.0 & 2.0 & 2.5 \end{bmatrix}$

Chaque ligne = un token
Chaque colonne = attention vers un autre token
Valeur élevée → forte attention



$A = softmax(QK^T / \sqrt{d_k})$

$A = \begin{bmatrix} 0.336 & 0.124 & 0.336 & 0.204 \\ 0.092 & 0.249 & 0.249 & 0.411 \\ 0.189 & 0.189 & 0.311 & 0.311 \\ 0.092 & 0.249 & 0.249 & 0.410 \end{bmatrix}$

- "this" regarde surtout → "this"(0.34) "smells" (0.34)
- "nice" regarde surtout → "good"(0.41)
- "smells" regarde surtout → "smells"(0.31) "good"(0.31)
- "good" regarde surtout → "good"(0.41)

Étape 4- Calcul de la sortie de l'attention

$Z = A \times V$

$Z = \begin{bmatrix} 1.408 & 0.992 & 1.664 & 1.000 \\ 1.163 & 1.671 & 1.920 & 1.001 \\ 1.311 & 1.311 & 1.933 & 1.000 \\ 1.163 & 1.669 & 1.918 & 1.000 \end{bmatrix}$

→ "this"
→ "nice"
→ "smells"
→ "good"

Chaque ligne = nouveau vecteur d'un token après avoir intégré le contexte des autres

Étape 5- Add & LayerNorm

$H = X + Z$

$H = \begin{bmatrix} 2.408 & 1.163 & 2.311 & 1.163 \\ 0.992 & 2.671 & 2.311 & 2.669 \\ 2.664 & 1.920 & 2.933 & 2.918 \\ 1.000 & 2.001 & 1.000 & 2.000 \end{bmatrix}$

Chaque token garde sa représentation originale ET absorbe le contexte des autres tokens



RISQUE





$$H_{\text{norm}} = \begin{bmatrix} -0.87 & -1.45 & -0.68 & -1.44 \\ -1.27 & 1.37 & -0.68 & 0.65 \\ 0.98 & -0.04 & 0.98 & 0.91 \\ 1.16 & 0.11 & -1.38 & -0.18 \end{bmatrix}$$

normalisation des valeurs
→ moyenne ≈ 0
→ variance ≈ 1

Étape 6– FFN (Feed Forward Network)

C'est un petit réseau de neurones appliqué sur chaque token $\mathcal{X}$ indépendamment

$$\text{FFN}(x) = \text{GELU}(xW_1)W_2$$



Permet de répondre à QUE SIGNIFIE ce lien ?

**Attention + FFN Les deux ensemble
permettent à BERT de comprendre le sens
profond du texte**

Étape 7– LayerNorm Finale

$$Y_{\text{norm}} = \begin{bmatrix} -1.21 & -1.47 & -1.38 & -1.44 \\ 1.31 & 1.43 & 1.38 & 1.41 \\ 0.21 & -0.02 & 0.00 & 0.01 \\ -0.31 & 0.06 & 0.00 & 0.02 \end{bmatrix}$$

Toutes les représentations sont
normalisées
→ prêtes à entrer dans la
couche suivante

SBERT – Sentence BERT

$$\text{Sentence Embedding} = \frac{1}{N} \sum_{i=1}^N h_i^{(\text{last layer})}$$

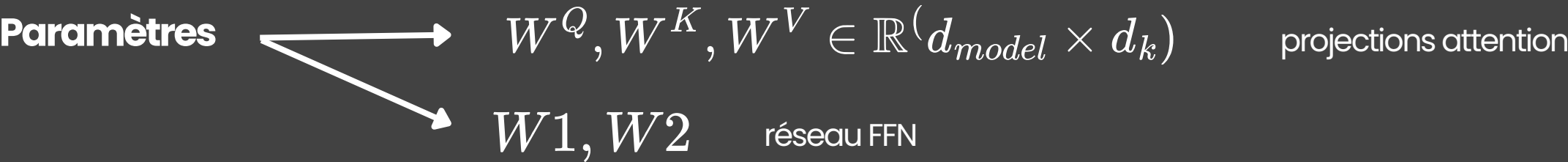


Un seul vecteur résume toute la phrase



BERT Base — Paramètres/Hyperparamètres principaux

Paramètres	Symbole	Valeur BERT	Impact
Dimension cachée	d_model	768	plus grand = représentations plus riches mais plus lent
Dimension des têtes	dk	64	contrôle la finesse de l'attention
Nombres de têtes et couches	h ; L	12	chaque tête capture un aspect différent plus de couches = abstractions plus profondes





$$X = E_{\text{token}} + E_{\text{position}} + E_{\text{segment}}$$

$E_{\text{token}}, E_{\text{position}}, E_{\text{segment}}$ sont des matrices apprises

$$X \in \mathbb{R}^{n \times d_{\text{model}}}$$



$$Q = XW^Q; K = XW^K; V = XW^V$$



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_i)W_O$$

$$\text{head}_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i)$$

- Chaque head capture des relations différentes
- Tout est entraîné conjointement (end-to-end)



$$Z = \text{LayerNorm}(X + \text{MultiHead}(X))$$

- Skip connection facilite la propagation du gradient



$$\text{FFN}(Z) = \max(0, ZW_1 + b_1)W_2 + b_2$$

- Appliqué indépendamment à chaque token
- Activation souvent GELU dans BERT réel

Merci

Pour votre attention